Day 2 Review: Tokens, Context and Latency

Module 1 (LLM Fundamentals), Unit 1.2

Support-bot estimate: checking the work

Scenario: 3,000 input tokens and 400 output tokens per request, priced at $1 per million input tokens and $5 per million output tokens. Time to first token is 1.0 s, generation speed is 50 tokens per second, and volume is 20,000 requests per month.

- **Cost per request:** 3,000 × $1/M = $0.003 for input, plus 400 × $5/M = $0.002 for output, so **$0.005 per request**.
- **Monthly cost:** 20,000 × $0.005 = **$100**.
- **Response time:** 400 ÷ 50 = 8 s of generation, plus 1 s to first token, so **9 s**.
- **Better-chosen context:** shrinking the article from 2,500 to 300 tokens cuts the input to 800 tokens. One request then costs $0.0008 + $0.002 = $0.0028, which is **$56 per month**, a saving of $44.

Two things worth noting

1. The simple timing model keeps the time at 9 s, but real systems also take less time to first token when the input is shorter.
2. After the change, output is about 71% of the cost, so the next lever for saving money is answer length, not input length.

The risk of better-chosen context is retrieving the wrong section, after which the bot answers confidently from the wrong text. Test accuracy before trusting the saving. Finding the right section is called retrieval, and it is the core problem of Module 4 (RAG).

1. Topics covered

Tokens vs words, why some text costs more, input vs output tokens, what fills the context window, context vs knowledge vs memory, why bigger windows aren't better, and latency.

2. Key concepts to remember

- Token counts depend on the model's tokenizer. Always count with the target model's tokenizer.
- Hindi used about 3.6× more tokens than the same English sentence in the test. Code and numbers also cost more per character.

- `cost = input_tokens × input_price + output_tokens × output_price`, and output is usually the pricier side.
- Input and output share one context window, and the model's own answer is appended to it.
- Training knowledge is fixed, context is temporary, and memory only exists if you build it. Chat apps re-send the history each time.
- More context isn't better. Facts in the middle get missed, and extra text dilutes focus.
- `total time ≈ time to first token + output tokens ÷ tokens per second`. Streaming improves how fast a response feels.

3. Code and exercises completed

Token counting across English, Hindi, code and numbers, cost projections for 10,000 requests, the input/output cost split, the response-time calculations, and the support-bot estimate.

4. Mistakes and misunderstandings to revisit

- The model *samples* from the probabilities. Picking the single most probable token is greedy decoding.
- The mechanism of memory: the app re-sends the history, and the model itself retains nothing.
- Code and numbers cost more because of symbols, short names and split digit groups, not only because they are rare.
- Fixes for slowness: list distinct ideas, including cutting output length and using a smaller model.

5. Quick knowledge check

Answer in one line each.

1. A model has a 4,000-token window. The prompt uses 3,200 tokens. What is the maximum answer length?
2. Why is a 200-token *input* usually less of a speed problem than a 200-token *output*?
3. Your app sends the same 2,000-token system prompt on every request. Name one thing that matters about this for cost, even without a formula.
4. True or false, with a reason: "A model with a 1M-token window will always answer better if I paste everything in."

6. Practical assignment

Pick a real piece of your own text, for example a README or a paragraph from your portfolio. Write a small script that takes a text and a model's prices and prints the token count, the cost per request, and the cost for 10,000 requests. Add a function that estimates response time from the output length. Keep the prices as variables at the top, since they change often. Then send the code for review, including its structure and edge cases.

7. Next topic

Day 3 of Module 1.

Still open from Day 1

- The knowledge-check quiz.
- The experiment of sending the same prompt five times and noting how the answers vary.